

ADA - Análisis y Diseño de Algoritmos, 2026-1

Tarea 2: Semanas 4 y 5

Para entregar el martes 10 de marzo de 2026

Problemas conceptuales a las 23:59 por BrightSpace

Problemas prácticos a las 23:59 en la arena de programación

Tanto los ejercicios como los problemas deben ser resueltos, pero únicamente las soluciones de los problemas deben ser entregadas. La intención de los ejercicios es entrenarlo para que domine el material del curso; a pesar de que no debe entregar soluciones a los ejercicios, usted es responsable del material cubierto en ellos.

Instrucciones para la entrega

Para esta tarea y todas las tareas futuras, la entrega de soluciones es *individual*. Por favor escriba claramente su nombre, código de estudiante y sección en cada hoja impresa entregada o en cada archivo de código (a modo de comentario). Adicionalmente, agregue la información de fecha y nombres de compañeros con los que colaboró; igualmente cite cualquier fuente de información que utilizó.

¿Cómo describir un algoritmo?

En algunos ejercicios y problemas se pide “dar un algoritmo” para resolver un problema. Una solución debe tomar la forma de un pequeño ensayo (es decir, un par de párrafos). En particular, una solución debe resumir en un párrafo el problema y cuáles son los resultados de la solución. Además, se deben incluir párrafos con la siguiente información:

- una descripción del algoritmo en castellano y, si es útil, pseudo-código;
- por lo menos un diagrama o ejemplo que muestre cómo funciona el algoritmo;
- una demostración de la corrección del algoritmo; y
- un análisis de la complejidad temporal del algoritmo.

Recuerde que su objetivo es comunicar claramente un algoritmo. Las soluciones algorítmicas correctas y descritas *claramente* recibirán alta calificación; soluciones complejas, obtusas o mal presentadas recibirán baja calificación.

Ejercicios

La siguiente colección de ejercicios, tomados del libro de Cormen et al. es para repasar y afianzar conceptos, pero no deben ser entregados como parte de la tarea.

14.1-2, 14.1-3, 14.2-1, 14.2-4, 14.4-1, 14.4-2, 14.4-5, 14.5-1.

Problemas conceptuales

1. Ejercicio 6-4: *Lightweight consulting business* (Kleinberg & Tardos, página 315).
2. Ejercicio 3.8: *Forward and backward substring* (Erickson, página 128).

Problemas prácticos

Hay cinco problemas prácticos cuyos enunciados aparecen a partir de la siguiente página.

A - Best Coalitions

Source file name: `best.py`

Time limit: x seconds

Envy Inc. is a joint stock company, that is, a company in which every stockholder legally owns shares of stock that account for some percentage of the total shares of stock of the company. Due to the global economic crisis, the management rules of Envy Inc. define a particular way for distributing last year's profit: if a stockholder owns more than half of the shares of stock, he/she wins the total profit. Nothing fancy so far in this wild world!

Nevertheless, there are situations in which no stockholder owns more than 50% of the shares of stock of the company. So, in order to gain some profit, stockholders are allowed to form *coalitions*, i.e., groups of stockholders. The participation of the coalition, share-wise, is equivalent to the sum of its stockholders' percentile participation. Hence, if a coalition has more than half of the shares of stock, its members win the totality of last years profit. Then, the members of the coalition receive a part of the profit proportional to their individual participation in the coalition.

For instance, let us assume there are 5 stockholders: *A*, *B*, *C*, *D* and *E*, owning 20%, 12%, 14%, 29% and 25% of the stock of the company, respectively. The stockholder *E* could form several winning coalitions. For example, if *E* were to form a coalition with *A* and *B*, he/she would get 43.86% of last year's profit. If *E* were to form a coalition with *B* and *C* instead, he/she would get 49.02% of last year's profit. On the other hand, *E* could not form a winning coalition with only *A*.

Your problem is, given a distribution of shares of stock of Envy Inc., and a stockholder, to determine the maximum percentage of the last year's profit that the given stockholder may win.

Input

The input consists of several test cases, each one defining a percentile distribution of shares of stock, and the index of a stockholder to determine his/her optimal participation. More precisely, each test case is defined by several input lines:

- the first line contains two integer values n ($1 \leq n \leq 100$) and x ($1 \leq x \leq n$), separated by a blank, representing the number of stockholders in Envy Inc. and the index of a stockholder to determine his/her optimal participation, respectively;
- each one of the following n lines has a single floating point value p_i , rounded to 2 decimal places, which represents the percentage of stock ownership of stockholder i ($1 \leq i \leq n$). The floating point delimiter is '.' (i.e., the dot). You can assume that $p_1 + \dots + p_n = 100$.

The end of the input is indicated by $n = x = 0$, an artificial case that must be ignored.

The input must be read from standard input.

Output

For each given case, output a single line with the corresponding answer. The answer should be formatted and approximated to two decimal places. The floating point delimiter must be '.' (i.e., the dot). The rounding applies towards the nearest *neighbor* unless both neighbors are equidistant, in which case the result is rounded up (e.g. 78.312 is rounded to 78.31; 78.566 is rounded to 78.57; 78.345 is rounded to 78.35, etc.).

The output must be written to standard output.

Sample Input	Sample Output
5 5	49.02
20.00	100.00
12.00	43.13
29.00	18.18
14.00	
25.00	
2 1	
56.87	
43.13	
2 2	
56.87	
43.13	
3 1	
10.00	
45.00	
45.00	
0 0	

B - Coffee Central

Source file name: `coffee.py`

Time limit: x seconds

Is it just a fad or is it here to stay? You're not sure, but the steadily increasing number of coffee shops that are opening in your hometown has certainly become quite a draw. Apparently, people have become so addicted to coffee that apartments that are close to many coffee shops will actually fetch higher rents.

This has come to the attention of a local real-estate company. They are interested in identifying the most valuable locations in the city in terms of their proximity to large numbers of coffee shops. They have given you a map of the city, marked with the locations of coffee shops. Assuming that the average person is willing to walk only a fixed number of blocks for their morning coffee, you have to find the location from which one can reach the largest number of coffee shops. As you are probably aware, your hometown is built on a square grid layout, with blocks aligned on north-south and east-west axes. Since you have to walk along streets, the distance between intersections (a, b) and (c, d) is $|a - c| + |b - d|$.

Input

The input contains several test cases. Each test case describes a city. The first line of each test case contains four integers dx , dy , n , and q . These are the dimensions of the city grid $dx \times dy$ ($1 \leq dx, dy \leq 1000$), the number of coffee shops n ($0 \leq n \leq 5 \cdot 10^5$), and the number of queries q ($1 \leq q \leq 20$). Each of the next n lines contains two integers x_i and y_i ($1 \leq x_i \leq dx$, $1 \leq y_i \leq dy$); these specify the location of the i -th coffee shop. There will be at most one coffee shop per intersection. Each of the next q lines contains a single integer m ($0 \leq m \leq 10^6$), the maximal distance that a person is willing to walk for a cup of coffee.

The last test case is followed by a line containing four zeros.

The input must be read from standard input.

Output

For each test case in the input, display its case number. Then display one line per query in the test case. Each line displays the maximum number of coffee shops reachable for the given query distance m followed by the optimal location. For example, the sample output shows that 3 coffee shops are within query distance 1 of the optimal location (3, 4), 4 shops are within query distance 2 of optimal location (2, 2), and 5 shops are within query distance 4 of optimal location (3, 1). If there are multiple optimal locations, pick the location that is furthest south (minimal positive integer y -coordinate). If there is still a tie, pick the location furthest west (minimal positive integer x -coordinate).

Follow the format of the sample output.

- a single integer giving the number of pennies collected

The output must be written to standard output.

Sample Input	Sample Output
<pre> 4 4 5 3 1 1 1 2 3 3 4 4 2 4 1 2 4 0 0 0 0 </pre>	<pre> Case 1: 3 (3,4) 4 (2,2) 5 (3,1) </pre>

C - The Poor Giant

Source file name: `giant.py`

Time limit: x seconds

On a table, there are n apples, the i -th apple has the weight $k + i$ ($1 \leq i \leq n$). Exactly one of the apples is sweet, lighter apples are all bitter, while heavier apples are all sour. The giant wants to know which one is sweet, the only thing he can do is to eat apples. He hates bitter apples and sour apples, what should he do?

For examples, $n = 4$, $k = 0$, the apples are of weight 1, 2, 3, 4. The giant can first eat apple #2.

- if #2 is sweet, the answer is #2
- if #2 is sour, the answer is #1
- if #2 is bitter, the answer might be #3 or #4, then he eats #3, he'll know the answer regardless of the taste of #3

The poor giant should be prepared to eat some bad apples in order to know which one is sweet. Let's compute the total weight of apples he must eat in all cases.

- #1 is sweet: 2
- #2 is sweet: 2
- #3 is sweet: $2 + 3 = 5$
- #4 is sweet: $2 + 3 = 5$

The total weights = $2 + 2 + 5 + 5 = 14$.

This is not optimal. If he eats apple #1, then he eats total weight of 1, $(1 + 3)$, $(1 + 3)$, $(1 + 3)$ when apple #1, #2, #3 and #4 are sweet respectively. This yields a solution of $1 + (1 + 3) + (1 + 3) + (1 + 3) = 13$, beating 14.

What is the minimal total weight of apples in all cases?

Input

The first line of input contains a single integer t ($1 \leq t \leq 100$), the number of test cases. The following t lines each contains a positive integer n and a non-negative k ($1 \leq n + k \leq 500$).

The input must be read from standard input.

Output

For each test case, output the minimal total weight in all cases as shown in the sample output.

The output must be written to standard output.

Sample Input	Sample Output
5	Case 1: 2
2 0	Case 2: 6
3 0	Case 3: 13
4 0	Case 4: 22
5 0	Case 5: 605
10 20	

D - Hippy Hopscotch

Source file name: hopscotch.py

Time limit: x seconds

The game of hopscotch involves chalk, sidewalks, jumping, and picking things up. Our variant of hopscotch involves money as well. The game is played on a square grid of dimension n : each grid location is labelled (p, q) where $0 \leq p < n$ and $0 \leq q < n$. Each grid location has on it a stack of between 0 and 100 pennies.

A contestant begins by standing at location $(0, 0)$. The contestant collects the money where he or she stands and then jumps either horizontally or vertically to another square. That square must be within the jumping capability of the contestant (say, k locations) and must have more pennies than those that were on the current square.

Given n , k , and the number of pennies at each grid location, compute the maximum number of pennies that the contestant can pick up before being unable to move.

Input

The input begins with a single positive integer on a line by itself indicating the number of the cases following, each of them as described below. This line is followed by a blank line, and there is also a blank line between two consecutive inputs.

- a line containing two integers between 1 and 100: n and k
- n lines, each with n numbers: the first line contains the number of pennies at locations $(0, 0), (0, 1), \dots, (0, n - 1)$; the next line contains the number of pennies at locations $(1, 0), (1, 1), \dots, (1, n - 1)$, and so on.

The input must be read from standard input.

Output

For each test case, the output must follow the description below. The outputs of two consecutive cases will be separated by a blank line.

- a single integer giving the number of pennies collected

The output must be written to standard output.

Sample Input	Sample Output
1	37
3 1	
1 2 5	
10 11 6	
12 12 7	

E - Ingredients

Source file name: `ingredients.py`

Time limit: x seconds

The chef of a restaurant aspiring for a Michelin star wants to display a selection of her signature dishes for inspectors. For this, she has allocated a maximum budget B for the cumulated cost, and she wants to maximize the cumulated prestige of the dishes that she is showing to the inspectors.



To measure the prestige of her dishes, the chef maintains a list of recipes, along with their costs and ingredients. For each recipe, a derived dish is obtained from a base dish by adding an ingredient. The recipe mentions two extra pieces of information: the cost of applying the recipe, on top of the cost of the base dish, and the prestige the recipe adds to the prestige of the base dish. The chef measures the prestige by her own units, called “prestige units”.

For example, a recipe list for making pizza looks like:

- `pizza_tomato pizza_base tomato 1 2`
- `pizza_classic pizza_tomato cheese 5 5`

Here, `pizza_base` is an *elementary dish*, a dish with no associated recipe, a dish so simple that its cost is negligible (set to 0) and its prestige also 0. The chef can obtain the *derived dish* `pizza_tomato` by adding the *ingredient* `tomato` to the base dish `pizza_base`, for a cost of 1 euro and a gain of 2 prestige units. A `pizza_classic` is obtained from a `pizza_tomato` by adding `cheese`, for an added cost of 5, and a prestige of 5 added to the prestige of the base dish; this means the total cost of `pizza_classic` is 6 and its total prestige is 7.

A signature dish selection could for instance include both a `pizza_tomato` and a `pizza_classic`. Such a selection would have cumulated total prestige of 9, and cumulated total cost of 7.

Armed with the list of recipes and a budget B , the chef wants to provide a signature dish selection to Michelin inspectors so that the cumulated total prestige of the dishes is maximized, keeping their cumulated total cost at most B .

Important Notes

- No dish can appear twice in the signature dish selection.
- Any dish that does not appear as a derived dish in any recipe is considered to be an elementary dish, with cost 0 and prestige 0.
- A dish can appear more than once as a resulting dish in the recipe list; if there is more than one way to obtain a dish, the one yielding the smallest total cost is always chosen; if the total costs are equal, the one yielding the highest total prestige should be chosen.
- The recipes are such that no dish D can be obtained by adding one or more ingredients to D itself.

Input

The input contains several test cases, each of them as described below.

- The first line consists of the budget B , an integer.
- The second line consists of the number N of recipes, an integer.

- Each of the following N lines describes a recipe, as the following elements separated by single spaces: the derived dish name (a string); the base dish name (a string); the added ingredient (a string); the added price (an integer); the added prestige (an integer).

Limits

- $0 \leq B \leq 10,000$;
- $0 \leq N \leq 1,000,000$;
- there can be at most 10,000 different dishes (elementary or derived);
- costs and prestiges in recipes are between 1 and 10,000 (inclusive);
- strings contain at most 20 ASCII characters (letters, digits, and ' _ ' only).

The input must be read from standard input.

Output

For each test case, the output must follow the description below.

The output should consist of two lines, each with a single integer. On the first line: the maximal cumulated prestige within the budget. On the second line: the minimal cumulated cost corresponding to the maximal cumulated prestige, necessarily less than or equal to the budget.

The output must be written to standard output.

Sample Input	Sample Output
15 6 pizza_tomato pizza_base tomato 1 2 pizza_cheese pizza_base cheese 5 10 pizza_classic pizza_tomato cheese 5 5 pizza_classic pizza_cheese tomato 1 2 pizza_salami pizza_classic salami 7 6 pizza_spicy pizza_tomato chili 3 1	25 15